706. Design HashMap

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	EASY	Solved: July 8, 202

Problem Summary

Design a HashMap without built-in hash table libraries. Implement put(key, value), get(key) (return -1 if absent), and remove(key).

Approach

Since keys are constrained to 0 <= key <= 10^6, use a direct-address int[] of size 1000001 indexed by key. Initialize all slots to -1 as the sentinel for "key not present".

Algorithm

1. Initialize int[] map of size 1000001, fill all with -1 using Arrays.fill().
2. put(key, value) → map[key] = value.
3. get(key) → return map[key].
4. remove(key) → map[key] = -1.

Key Insights

- Direct extension of LC 705 — swap boolean[] for int[], swap false sentinel for -1.
- -1 is the perfect sentinel because the problem guarantees get() returns -1 for missing keys, and values are non-negative.
- Java initializes int[] to 0 by default — **not** -1 — so Arrays.fill() is mandatory here, unlike 705.

Points to Remember

1. **705 vs 706**: boolean[] + false sentinel → int[] + -1 sentinel. Same pattern, one step up.
2. Arrays.fill(arr, -1) is essential — never assume int[] defaults to -1.
3. **Sentinel value rule**: pick a value that can never be a valid answer — here -1 works since values ≥ 0.
4. For a real HashMap interview follow-up: buckets array + chaining (LinkedList per bucket) + key % capacity for hash function.

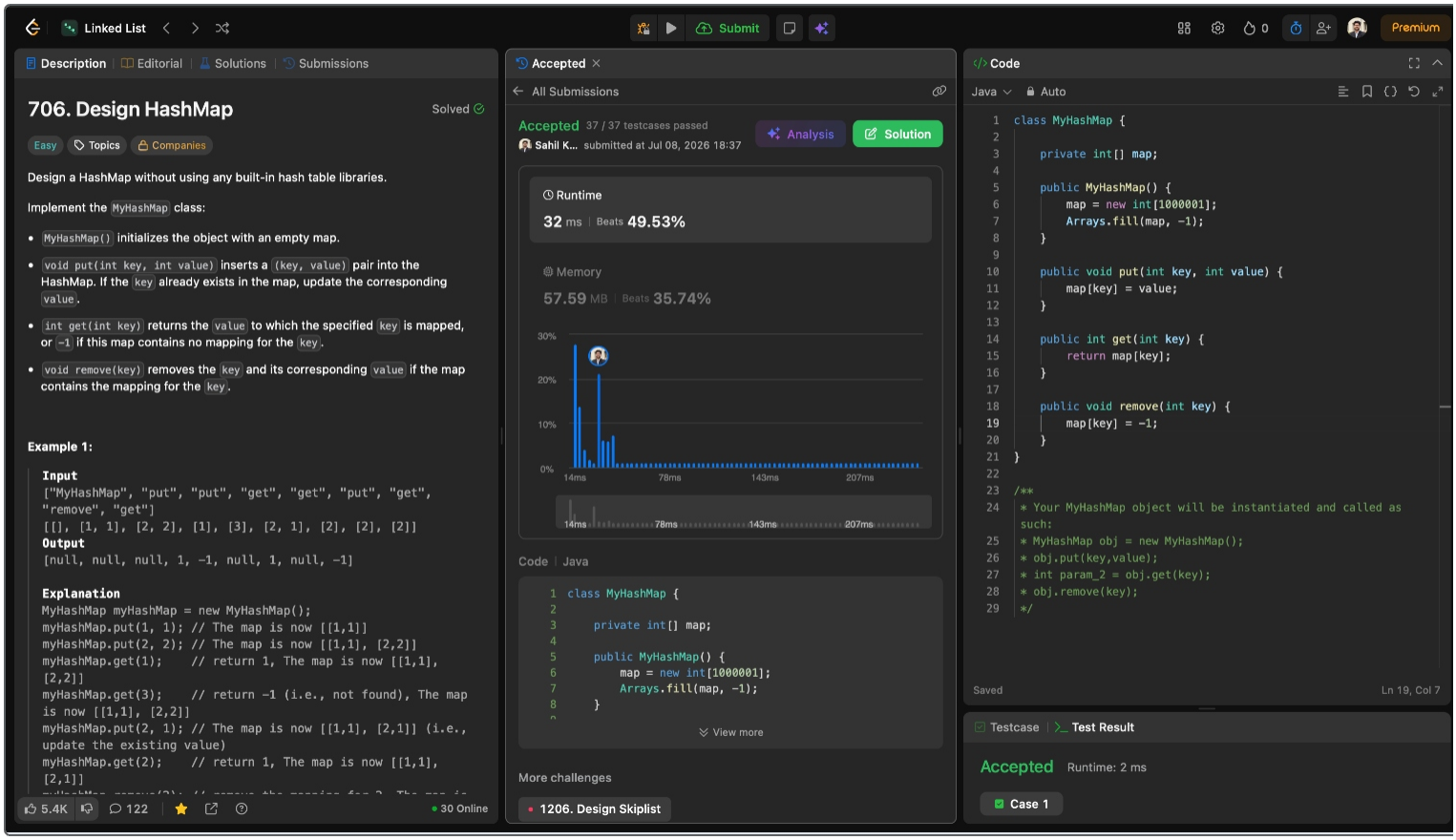
Observations

- put() naturally handles updates — overwriting map[key] replaces old value.
- No collision handling needed — key IS the index (direct addressing).
- Real HashMap would use array of LinkedLists + key % capacity for arbitrary key ranges.

Time Complexity & Space Complexity

O(1) — all operations are direct array access.

O(1) — fixed array of 1000001, independent of operations.


Linked List
Submit
Premium
Description | Editorial | Solutions | Submissions
706. Design HashMap
Solved
Easy
Topics
Companies
Design a HashMap without using any built-in hash table libraries.
Implement the MyHashMap class:
MyHashMap() initializes the object with an empty map.
void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.
Example 1:
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]
Explanation
MyHashMap myHashMap = new MyHashMap();
myHashMap.put(1, 1); // The map is now [[1,1]]
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]
myHashMap.get(1); // return 1, The map is now [[1,1], [2,2]]
myHashMap.get(3); // return -1 (i.e., not found), The map is now [[1,1], [2,2]]
myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing value)
myHashMap.get(2); // return 1, The map is now [[1,1], [2,1]]
5.4K
122
30 Online
Accepted
All Submissions
Accepted 37 / 37 testcases passed
Sahil K... submitted at Jul 08, 2026 18:37
Analysis
Solution
Runtime
32 ms | Beats 49.53%
Memory
57.59 MB | Beats 35.74%
30%
20%
10%
0%
14ms
78ms
143ms
207ms
Code | Java
class MyHashMap {
private int[] map;
public MyHashMap() {
map = new int[1000001];
Arrays.fill(map, -1);
}
View more
More challenges
1206. Design Skiplist
Code
Java
Auto
class MyHashMap {
private int[] map;
public MyHashMap() {
map = new int[1000001];
Arrays.fill(map, -1);
}
public void put(int key, int value) {
map[key] = value;
}
public int get(int key) {
return map[key];
}
public void remove(int key) {
map[key] = -1;
}
}
/**
* Your MyHashMap object will be instantiated and called as such:
* MyHashMap obj = new MyHashMap();
* obj.put(key,value);
* int param_2 = obj.get(key);
* obj.remove(key);
*/
Saved
Ln 19, Col 7
Testcase | Test Result
Accepted Runtime: 2 ms
Case 1